Lecture 4a: Neural Operator methods

Chris Budd and Aengus Roberts[1]

[1]University of Bath and KE Hub

Ai4Sci, October 2026

Some papers/books to look at

- Courant and Hilbert *Methods of mathematical physics Volumes 1,2*
- Stuart et. al. *Fourier Neural Operator for PDEs*
- Kovachi et. al. *Operator learning: algorithms and analysis*
- Boullé and Townsend *Learning elliptic PDEs*
- Halko et. al. *Finding structure with randomness*

Motivation: Solution Operators

Have studied using PINNS and DRMs to solve PDE problems of the form

$$u_t = F(\mathbf{x}, u, \nabla u, \nabla^2 u) \quad \text{with BC},$$

$$u(0, x) = u_0(x)$$

At time T we have the solution $u_T(x) \equiv u(x, T)$.

Solution $u(x, t)$ for all x and t is obtained by minimising a function directly associated with the PDE eg. residual.

Neural Operator methods take a different approach

- Consider u_T as a function F of u_0. $u_T = F(u_0)$
- F is an operator mapping one infinite dimensional function space to another $F : A \to B$. eg. $A, B = H^1(\Omega)$
- Train a Neural Operator NN to approximate this operator note infinite dimensions
- Train it by generating a (large) set of solution pairs (u_0^i, u_T^i)

Can generate solution pairs using a (conventional) numerical method eg. Finite Element, Pseudo-Spectral, Symplectic.

eg. ERA5 data for 24 hour weather forecasts.

Example 1: Blind channel identification

- Have an $n \times n$ matrix B
- Have a random set of N vectors $\mathbf{x}_i \in R^n$
- Compute the N (noisy) matrix vector products

$$B\,\mathbf{x}_i = \mathbf{y}_i \in R^n$$

Challenge: Construct the matrix B from the set of N solution pairs $(\mathbf{x}_i, \mathbf{y}_i)$

Methodology:

- Let $X = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N]$, $Y = [\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_N]$
- Find $B = Y\,X^+$ using the SVD/Moore-Penrose pseudo-inverse.

Example 2: A linear ODE system

Consider the linear ODE

$$\frac{d\mathbf{u}}{dt} = A\,\mathbf{u}, \quad \mathbf{u}(0) = \mathbf{u}_0, \quad \mathbf{u} \in R^n.$$

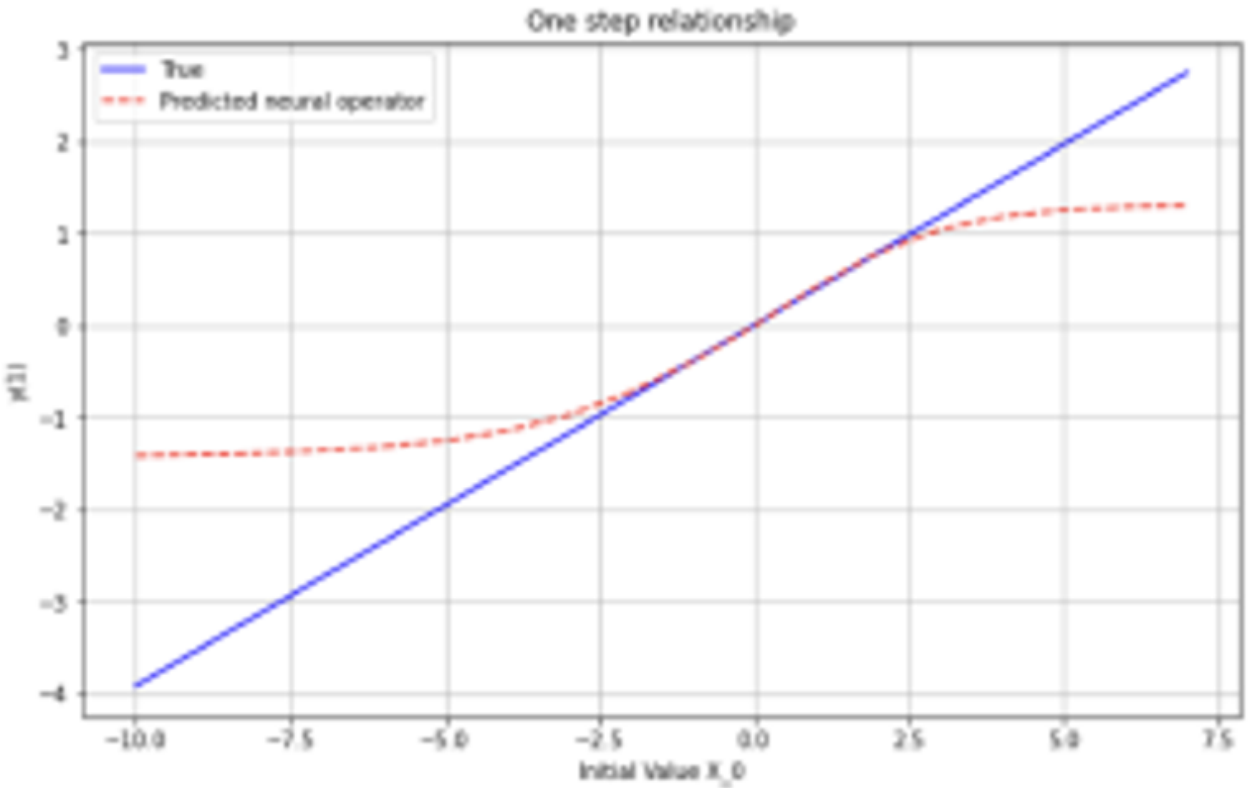
Solution

$$\mathbf{u}(T) = e^{A\,T}\mathbf{u}_0 \equiv B\,\mathbf{u}_0$$

$$B \equiv e^{AT} = I + AT + \frac{A^2T^2}{2!} + \frac{A^3T^3}{3!} + \ldots$$

Properties of the solution operator

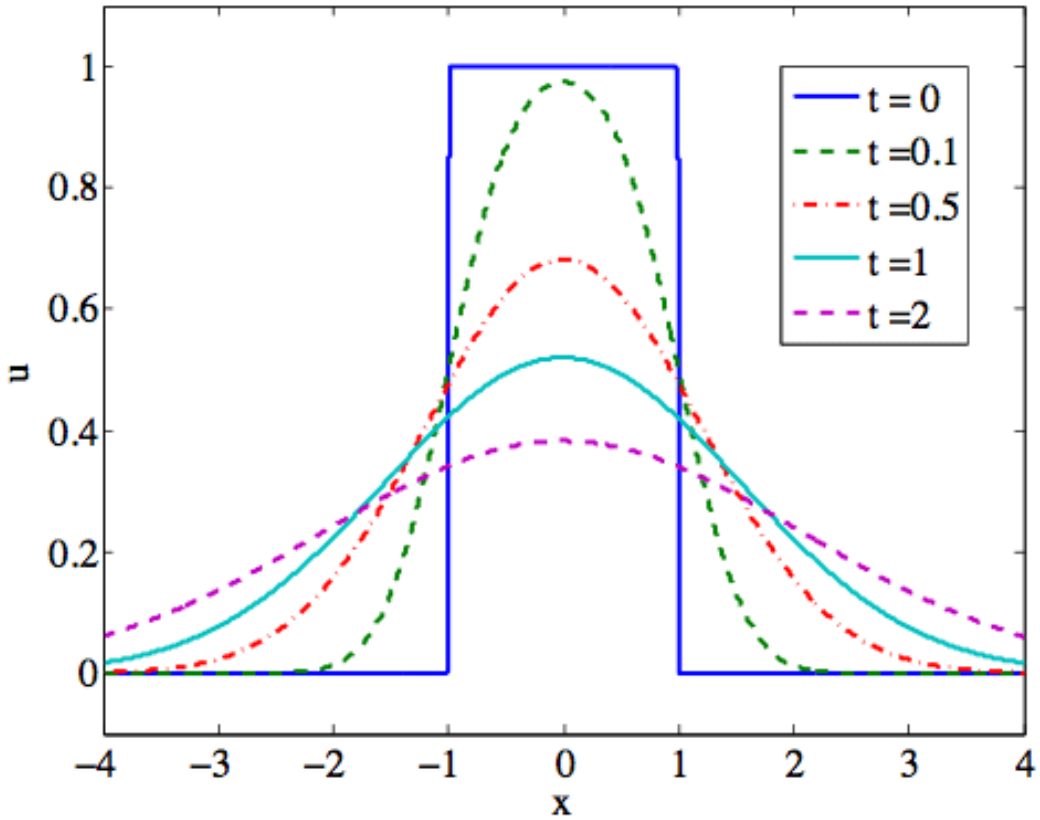
- Operator B is linear
- Operator is continuous over any subset of R^n
- Can easily learn the matrix B from data pairs if we assume that the operator is linear in advance!
- If we learn B from a subset of the data pairs then we can extrapolate this to ALL data pairs
- This is NOT true if don't make the linearity assumption. Many NN methods will locally approximate the operator to be linear, but will not give this as a global approximation.


One step relationship
True
Predicted neural operator
y[1]
Initial Value X_0
3
2
1
0
−1
−2
−3
−4
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5

Example 3: Parabolic PDEs

Consider the linear heat equation with a source

$$u_t = u_{xx} + f(x), \quad x \in [0, 2\pi], \quad u(0, x) = u_0(x), \quad \textit{periodicBC}$$



Solution using operators

We can express $u(x,T)$ in terms of convolutional integral operators:

$$u_T = u(x,T) \equiv F(u_0) = G * u_0 + H * f$$

$$\equiv \int_0^{2\pi} G(x-y,T)u_0(y)\,dy + \int_0^{2\pi} H(x-y,T)f(y)\,dy$$

Operators G, H act on the infinite dimensional space $L^2[0,2\pi]$.

Can find $G(z,T)$ and $H(z,T)$ **explicitly** using a Fourier series.

This methodology motivates the construction of the Fourier Neural Operator (FNO)

The FNO: In general

The FNO architecture is based on the process of solving the linear heat equation, but also works for **nonlinear problems.** The FNO constructs a 'Neural Map' Ψ acting on u_0 parametrised by θ as follows:

$$\Psi(u_0,\theta)_{FNO} \equiv Q \circ \mathcal{L}_L \circ \ldots \circ \mathcal{L}_2 \circ \mathcal{L}_1 \circ P(u_0) \approx u_T = F(u_0)$$

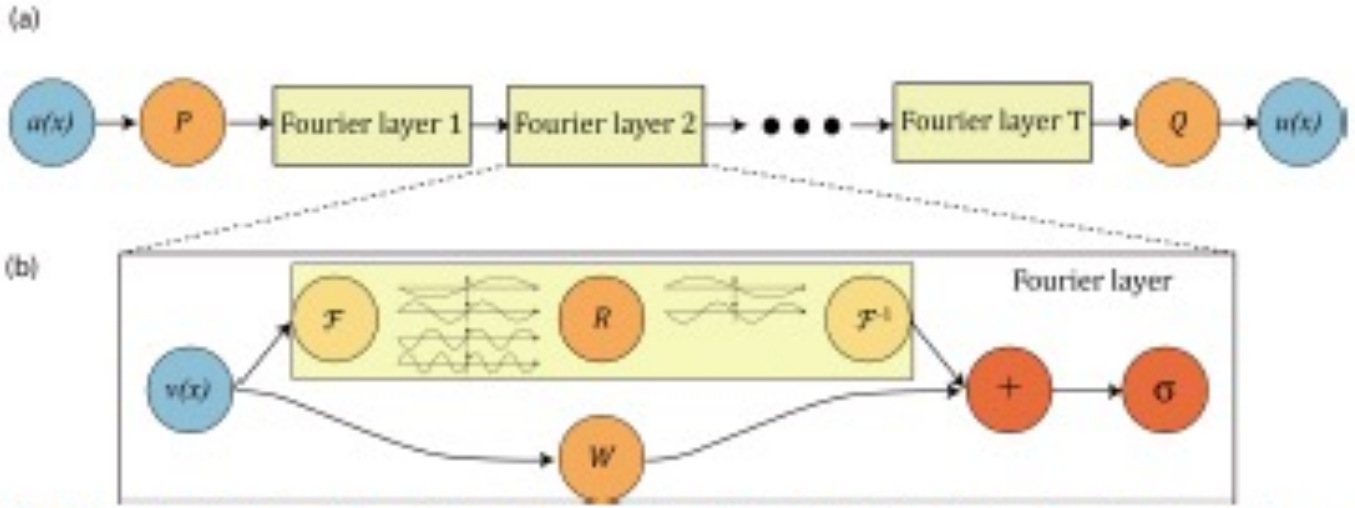
$$\mathcal{L}_n(v)(x,\theta) = \sigma(W_n v(x) + b_n + K_n(v))$$

Here W is a pointwise linear local map. $K(v)$ is a global convolutional integral operator, kernal $G_n(\theta)$. Evaluate Kv using an FFT via

$$FFT(K_n(v)) = FFT(G_n)\, FFT(v).$$

FFT restricted to M modes. Nonlinearity and higher order modes introduced via the activation function σ.

Figure from FNO paper



(a) The full architecture of neural operator: start from input a. 1. Lift to a higher dimension channel space by a neural network P. 2. Apply four layers of integral operators and activation functions. 3. Project back to the target dimension by a neural network Q. Output u. **(b) Fourier layers**: Start from input v. On top: apply the Fourier transform $\mathcal{F}$; a linear transform R on the lower Fourier modes and filters out the higher modes; then apply the inverse Fourier transform $\mathcal{F}^{-1}$. On the bottom: apply a local linear transform W.

Figure 2: **top:** The architecture of the neural operators; **bottom:** Fourier layer.

FNO in detail

- Input $u_0(x) \in \mathcal{U}$, output $\psi(u_0, \theta) \approx u_T = F(u_0)$ are functions on $x \in D \subset R^d$
- Assume have access to pointwise observations of u_0 only at points in $x_i \in D_j \subset D$. Output of the FNO does not depend on D_j: super-resolution
- Lift u_0 to a higher dimensional representation $v_0(x) = P(u_0(x))$ by a shallow NN.
- Calculate a series of updates $v_n \to v_{n+1}$ via the local W_n and global (integral) K_n operators:

$$v_{n+1}(x) = \sigma\left(W_n v_n(x) + (K_n(\theta)v_n))(x)\right).$$

- For example $\sigma = ReLU$: This introduces nonlinearity into the map in a slightly uncontrolled way
- Project $v_L \to Q(v_L) = \psi(u_0, \theta) \approx u_T$
- Learn P, Q, W_n, K_n from the data pairs

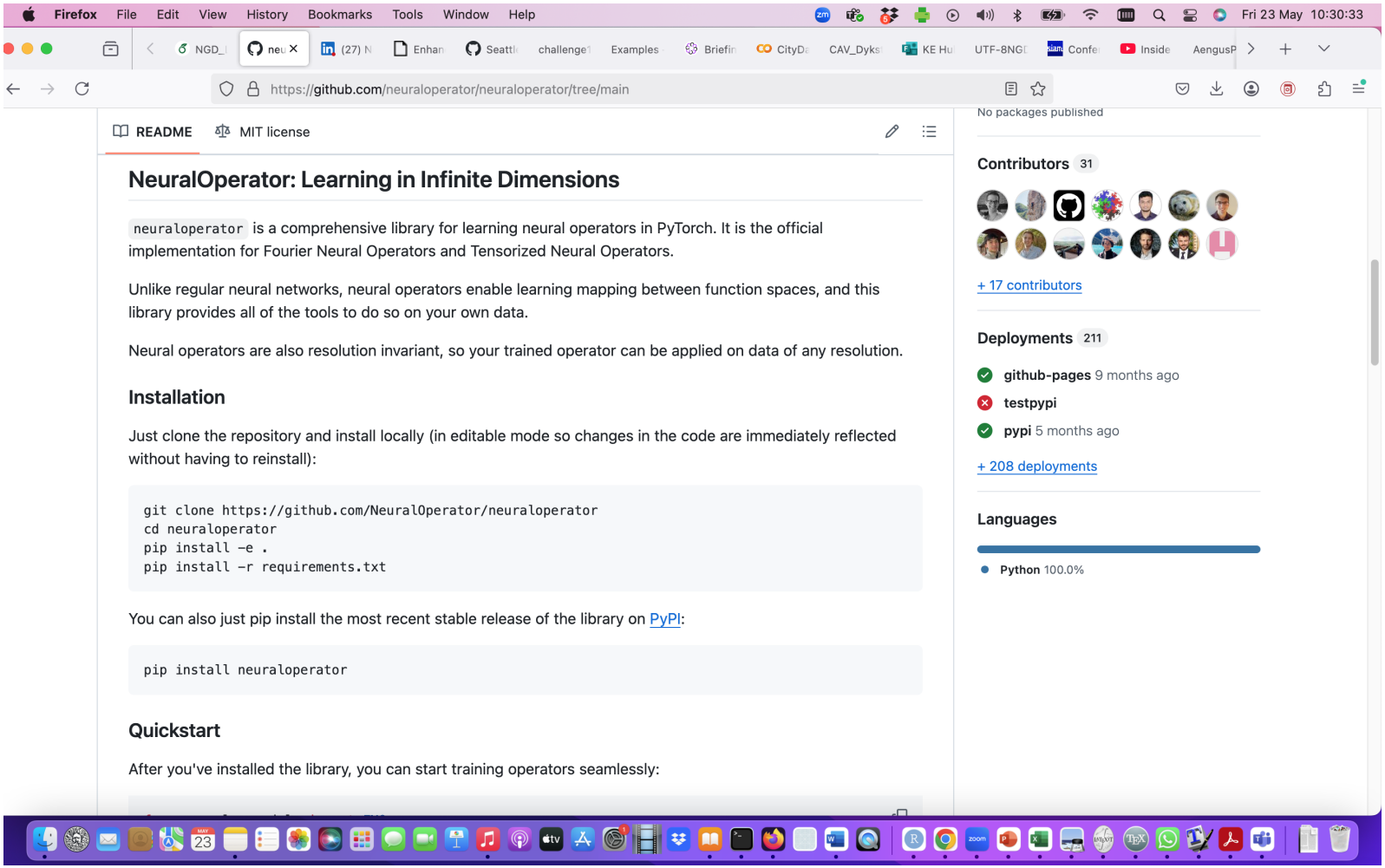
Training

- Assume a large number of inputs $u_0^j \in \mathcal{U}, \quad j = 1 \ldots N$
- In the original FNO paper take u_0^j as an i.i.d sequence from $\mathcal{U}$.
- Construct pairs (u_0^j, u_T^j) using an accurate solver eg. pseudo-spectral method
- FNO paper takes $N = 1000$ training, and 200 verification, pairs. Adam optimiser to find parameters θ via:

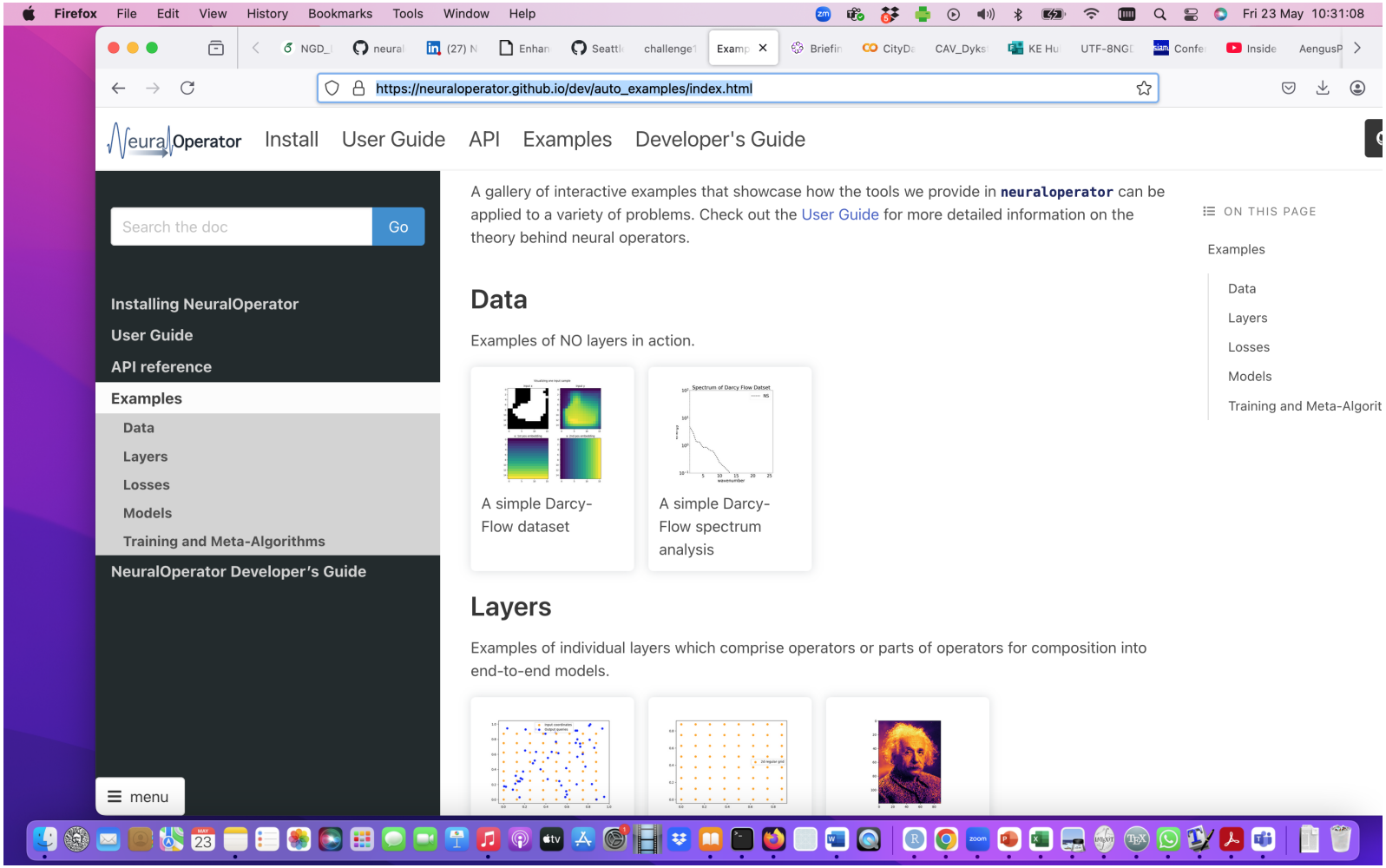
$$min_\theta E_{u\sim\mu}[\|\psi(u_0^j, \theta) - u_T^j\|]$$

- Can significantly improve training by a more careful selection of input and output pairs [Liu, B, et. al.]

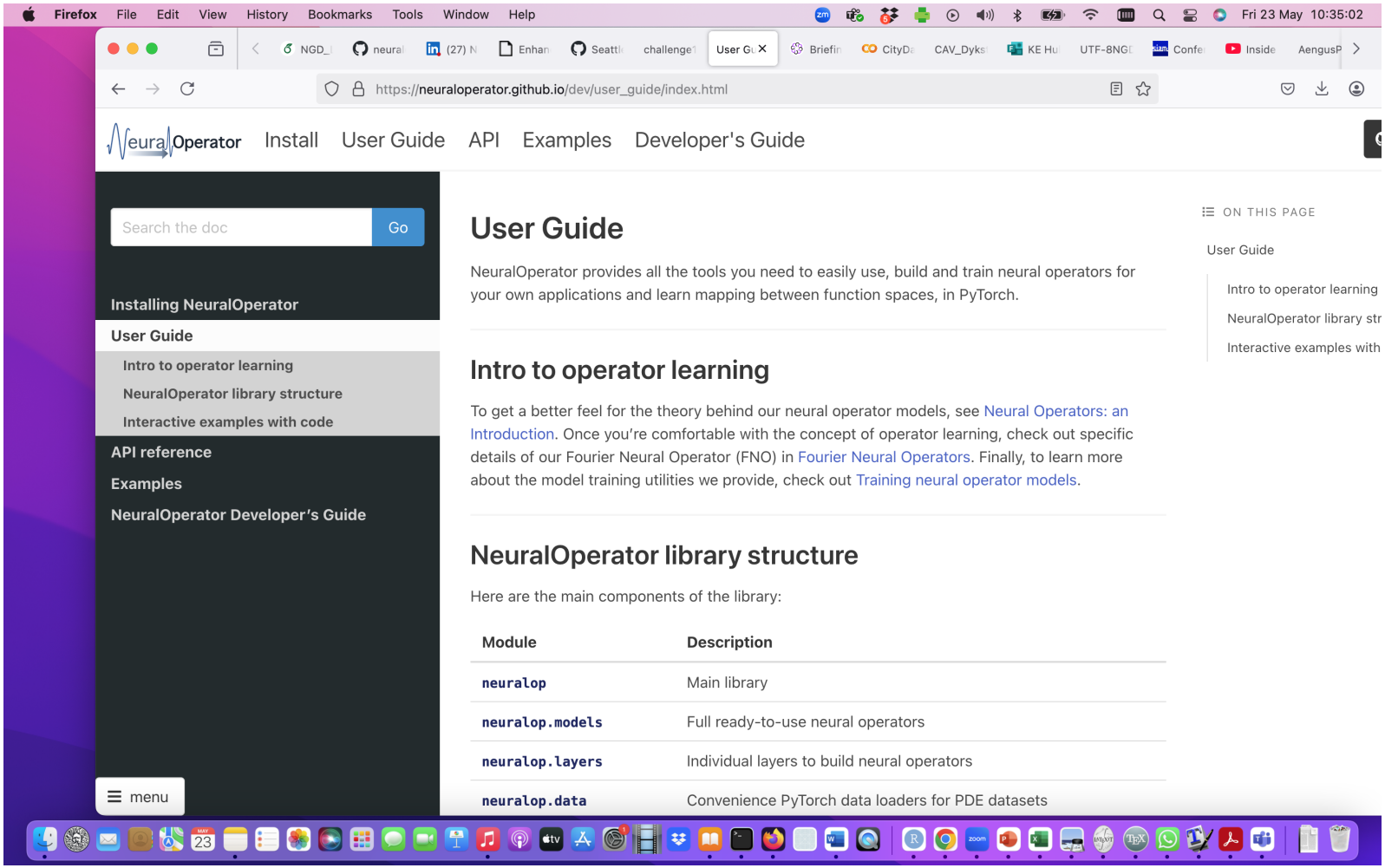
FNO online 1


README
MIT license
NeuralOperator: Learning in Infinite Dimensions
neuraloperator is a comprehensive library for learning neural operators in PyTorch. It is the official implementation for Fourier Neural Operators and Tensorized Neural Operators.
Unlike regular neural networks, neural operators enable learning mapping between function spaces, and this library provides all of the tools to do so on your own data.
Neural operators are also resolution invariant, so your trained operator can be applied on data of any resolution.
Installation
Just clone the repository and install locally (in editable mode so changes in the code are immediately reflected without having to reinstall):
git clone https://github.com/NeuralOperator/neuraloperator
cd neuraloperator
pip install -e .
pip install -r requirements.txt
You can also just pip install the most recent stable release of the library on PyPI:
pip install neuraloperator
Quickstart
After you've installed the library, you can start training operators seamlessly:
No packages published
Contributors 31
+ 17 contributors
Deployments 211
github-pages 9 months ago
testpypi
pypi 5 months ago
+ 208 deployments
Languages
Python 100.0%

FNO online 2


https://neuraloperator.github.io/dev/auto_examples/index.html
NeuralOperator
Install
User Guide
API
Examples
Developer's Guide
Search the doc
Go
Installing NeuralOperator
User Guide
API reference
Examples
Data
Layers
Losses
Models
Training and Meta-Algorithms
NeuralOperator Developer's Guide
menu
A gallery of interactive examples that showcase how the tools we provide in neuraloperator can be applied to a variety of problems. Check out the User Guide for more detailed information on the theory behind neural operators.
ON THIS PAGE
Examples
Data
Layers
Losses
Models
Training and Meta-Algorit
Data
Examples of NO layers in action.
A simple Darcy-Flow dataset
A simple Darcy-Flow spectrum analysis
Layers
Examples of individual layers which comprise operators or parts of operators for composition into end-to-end models.

FNO online 3

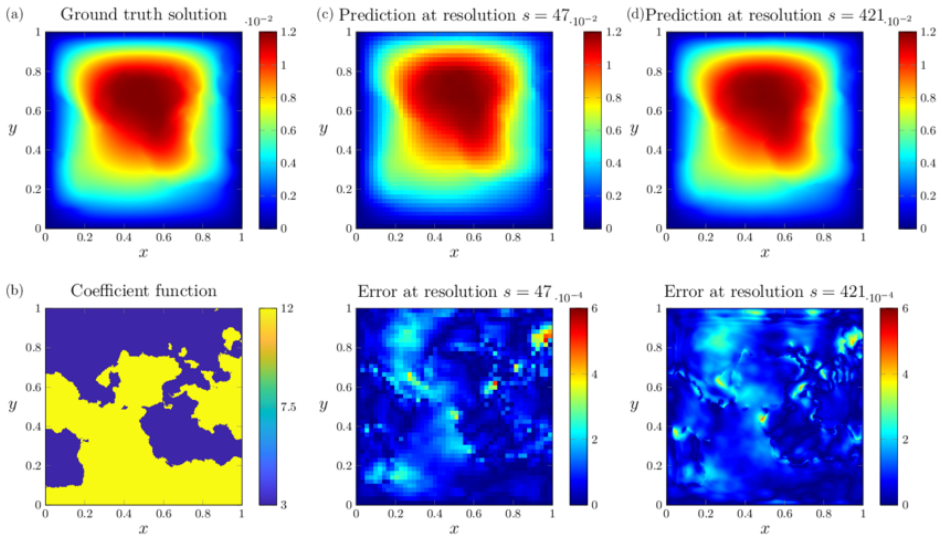

https://neuraloperator.github.io/dev/user_guide/index.html
NeuralOperator
Install
User Guide
API
Examples
Developer's Guide
Search the doc
Go
Installing NeuralOperator
User Guide
Intro to operator learning
NeuralOperator library structure
Interactive examples with code
API reference
Examples
NeuralOperator Developer's Guide
menu
User Guide
NeuralOperator provides all the tools you need to easily use, build and train neural operators for your own applications and learn mapping between function spaces, in PyTorch.
Intro to operator learning
To get a better feel for the theory behind our neural operator models, see Neural Operators: an Introduction. Once you're comfortable with the concept of operator learning, check out specific details of our Fourier Neural Operator (FNO) in Fourier Neural Operators. Finally, to learn more about the model training utilities we provide, check out Training neural operator models.
NeuralOperator library structure
Here are the main components of the library:
Module
Description
neuralop
Main library
neuralop.models
Full ready-to-use neural operators
neuralop.layers
Individual layers to build neural operators
neuralop.data
Convenience PyTorch data loaders for PDE datasets
ON THIS PAGE
User Guide
Intro to operator learning
NeuralOperator library str
Interactive examples with

Example 4: The Darcy Problem

The Darcy problem relates a permeability $a(x)$ to a velocity field $u(x)$

$$-\nabla \cdot (a(x)\nabla u) = f(x) \quad x \in \Omega, \quad u = 0 \quad x \in \partial\Omega.$$

This induces a (nonlinear) map $F : a \to u, \quad F : L^2(\Omega) \to H_0^1(\Omega)$



We can approximate the map F using the Finite Element Method

$$u(x) \approx U(x) = \sum U_i \, \phi_i(x).$$

$$-\nabla \cdot (a(x) \nabla u) = f \implies \int a(x) \nabla u(x) \cdot \nabla \phi_i(x) \, dx = \int f(x) \phi_i(x) \, dx \equiv f_i$$

Giving the linear system

$$A\mathbf{U} = \mathbf{f}, \quad \mathbf{U}_i = U_i, \quad \mathbf{f}_i = f_i, \quad A_{ij} = \int a(x) \, \nabla \phi_i \cdot \nabla \phi_j \, dx.$$

Hence we can approximate the nonlinear map F via:

$$\mathbf{U} = A^{-1} \, \mathbf{f}$$

And ...

We can LEARN this map by

- Doing lots of finite element calculations to find solution pairs $(a^j(x), u^j(x))$
- Learn the operator F between these pairs using an FNO

Convergence [Kovachi et. al.]

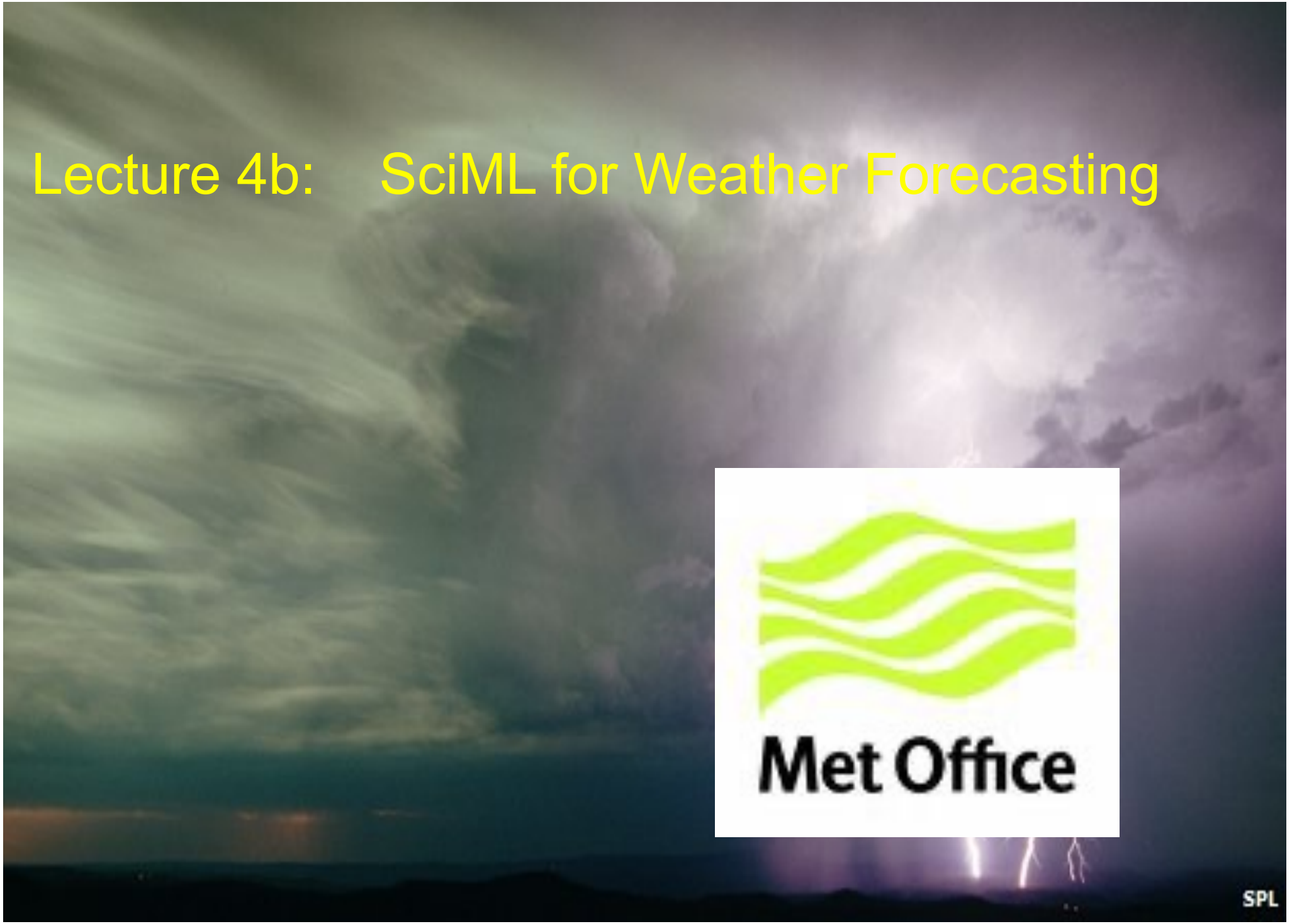
- FNO and similar methods such as DeepONet can approximate a wide variety of operators
- Assume that input space $\mathcal{U}$ is a separable Banach space and the map F is compact (eg. an integral operator)
- Prove convergence on any finite dimensional set using the universal approximation theorem
- Take an appropriate limit (approximation theory of Banach spaces which applies to the sets over which PDEs are typically formulated)

Areas for improvement and research on FNO

- Generating a good training set is crucial and can be slow. How to make it good and fast?
- FNO struggles away from the training set. This is OK for MCMC emulators for UQ.
- Observe poor conservation laws at the moment
- Need to broaden its scope and extend the theorems on its convergence
- NONE of this theory applies, for example, to the nonlinear heat equation

$$u_t = u_{xx} + u^2.$$

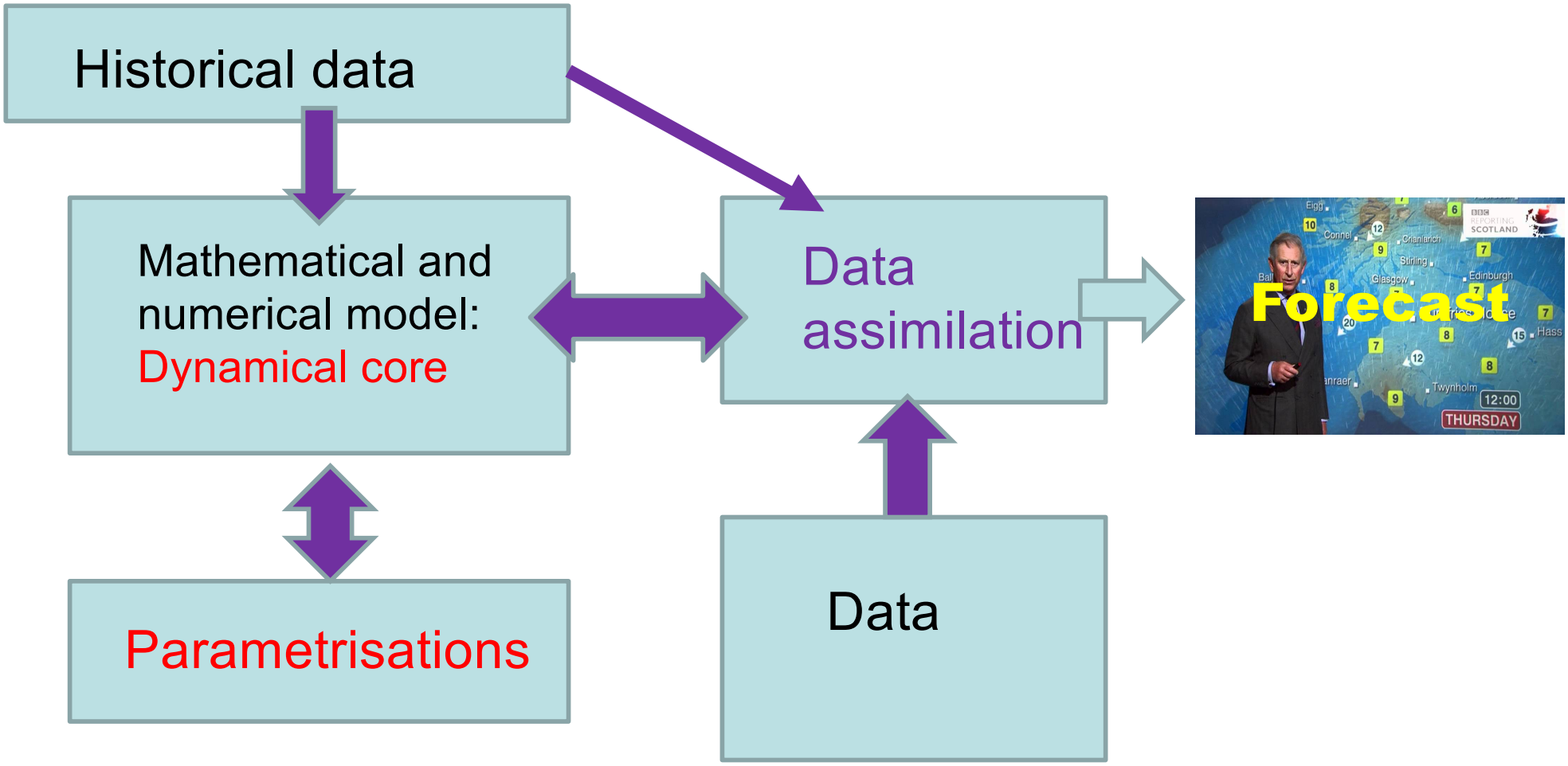
Lecture 4b: SciML for Weather Forecasting



Some papers

- GraphCast (2023)
- Neur GCM (2024)
- Aardvark (2025)
- Morcrette, B et. al. :*Scale-Aware Parameterization of Cloud Fraction and Condensate for a Global Atmospheric Model Machine-Learned From Coarse-Grained Kilometer-Scale Simulations*
- Shipway, B. et. al. *Blending machine learning and physics-based approaches for weather and climate: a typology*

Weather forecasting process



Areas where ML is used in weather forecasting

1. Post-processing results eg. down scaling
2. Parametrisations eg. clouds
3. Dynamical core/physics: GraphCast, NeurGCM
4. Finding the climatology and regularization: FastNet
5. End-to-end/data only: Aardvark

Complex interrelated processes described by differential equations

Basic equations: Navier-Stokes which describe the weather

$$\frac{Du}{Dt} + 2f \times u + \frac{1}{\rho}\nabla p + g = \nu\nabla^2 u,$$

Motion

$$\frac{\partial \rho}{\partial t} + \nabla.(\rho u) = 0,$$

Density

$$C\frac{DT}{Dt} - \frac{RT}{\rho}\frac{D\rho}{Dt} = \kappa_h \nabla^2 T + S_h + LP,$$

Temperature

$$\frac{Dq}{Dt} = \kappa_q \nabla^2 q + S_q - P,$$

Moisture

$$p = \rho RT.$$

Pressure

For climate add in ice, CO2, ocean currents, vegetation, …

Traditional methods ie. a highly accurate PDE solver (FEM/Finite Volume/Spectral) plus data assimilation (pre and post the forecast where possible)

Lead to ….

Data sets you can use to train Sci ML systems

- Reanalysis:

ERA5 (1979-2017 Weather predictions based on HRES forecasts at ECMWF)

- UKCP18. (Climate till 2050, based on HadGEM3)

ECMWF Reanalysis v5 (ERA5)

ew Datasets search

ERA5 is the fifth generation ECMWF atmospheric reanalysis of the global climate covering the period from January 1940 to present. ERA5 is produced by the Copernicus Climate Change Service (C3S) at ECMWF.

ERA5 provides hourly estimates of a large number of atmospheric, land and oceanic climate variables. The data cover the Earth on a 31km grid and resolve the atmosphere using 137 levels from the surface up to a height of 80km. ERA5 includes information about uncertainties for all variables at reduced spatial and temporal resolutions.

ERA5 is available on:

- Single levels
- Pressure levels: 1000/975/950/925/900/875/850/825/800/775/750/700/650/600/550/500/450/400/350/300/250/225/200/175/150/125/100/70/50/30/20/10/7/5/3/2/1
- Potential temperature levels:

Explore this dataset:

Climate Data Store ›

Access to the data portals and their features depends on who you are

View licence

Big claim weather forecasting papers which make use of these data sets

GraphCast (2023)

Aardvark (2025)

FastNET (2026)

Methodology: GNN/Neural operator based methods

Trained on ERA5 data and satellite data

Very rapid prediction times, but expensive to train

Firefox File Edit View History Bookmarks Tools Window Help Wed 28 May 08:50:32

www.nature.com/articles/d41586-023-03552-y

water discussion

nature

View all journals Search Log in

Explore content About the journal Publish with us Subscribe Sign up for alerts RSS feed

nature › news › article

NEWS | 14 November 2023

DeepMind AI accurately forecasts weather — on a desktop computer

The machine-learning model takes less than a minute to predict future weather worldwide more precisely than other approaches.

By Carissa Wong

You have full access to this article via **JISC Springer Compact (publishing + reading)**.

Cell Signaling TECHNOLOGY

Cancer Signaling Pathways & Diagrams

Download



File Edit View History Bookmarks Tools Window Help
file:///Users/mascjb/Downloads/science.adi2336.pdf
1416 (1 of 6) 110%

Check for updates

WEATHER FORECASTING

Learning skillful medium-range global weather forecasting

Remi Lam[1]*†, Alvaro Sanchez-Gonzalez[1]*†, Matthew Willson[1]*†, Peter Wirnsberger[1]†, Meire Fortunato[1]†, Ferran Alet[1]†, Suman Ravuri[1]†, Timo Ewalds[1], Zach Eaton-Rosen[1], Weihua Hu[1], Alexander Merose[2], Stephan Hoyer[2], George Holland[1], Oriol Vinyals[1], Jacklynn Stott[1], Alexander Pritzel[1], Shakir Mohamed[1]*, Peter Battaglia[1]*

Global medium-range weather forecasting is critical to decision-making across many social and economic domains. Traditional numerical weather prediction uses increased compute resources to improve forecast accuracy but does not directly use historical weather data to improve the underlying model. Here, we introduce GraphCast, a machine learning–based method trained directly from reanalysis data. It predicts hundreds of weather variables for the next 10 days at 0.25° resolution globally in under 1 minute. GraphCast significantly outperforms the most accurate operational deterministic systems on 90% of 1380 verification targets, and its forecasts support better severe event prediction, including tropical cyclone tracking, atmospheric rivers, and extreme temperatures. GraphCast is a key advance in accurate and efficient weather forecasting and helps realize the promise of machine learning for modeling complex dynamical systems.

It is 05:45 UTC (coordinated universal time) in mid-October 2022 in Bologna, Italy, at the recently opened high-performance computing facility of the European Centre for Medium-Range Weather Forecasts (ECMWF). For the past several hours, the Integrated Forecasting System (IFS) has been running sophisticated calculations to forecast Earth's weather over the next days and weeks, and its first predictions have just begun to be disseminated to users. This process repeats every 6 hours, every day, to supply the world with the most accurate weather forecasts available.

The IFS, and modern weather forecasting more generally, are triumphs of science and

days ahead—a possibility that was unthinkable even a few decades ago.

But while traditional NWP scales well with compute, capitalizing on the vast amount of historical weather data to improve accuracy is not straightforward. Rather, NWP methods are improved by highly trained experts innovating better models, algorithms, and approximations, which can be a time-consuming and costly process.

Machine learning–based weather prediction (MLWP)—wherein forecast models are trained from historical data, including observations and analysis data—offers an alternative to traditional NWP. MLWP has the potential to improve

tively weak, for example, in subseasonal heat wave prediction (*3*) and precipitation nowcasting from radar images (*4*–*7*), where accurate equations and robust numerical methods are not as available.

In medium-range weather forecasting—the prediction of atmospheric variables up to 10 days ahead—NWP-based systems such as the IFS are still most accurate. The top deterministic operational system in the world is ECMWF's high-resolution forecast (HRES), a configuration of IFS that produces global 10-day forecasts at 0.1° latitude and longitude resolution, in around an hour (*8*). However, over the past several years, MLWP methods for medium-range forecasting trained on reanalysis data have been steadily advancing, facilitated by benchmarks such as WeatherBench (*8*). Deep learning architectures based on convolutional neural networks (*9*–*11*) and Transformers (*12*) have shown promising results at latitude and longitude resolutions coarser than 1.0°, and recent works—which use graph neural networks (GNNs), Fourier neural operators, and Transformers (*13*–*16*)—have reported performance that begins to rival IFS's at 1.0° and 0.25° for a handful of variables and lead times up to 7 days.

GraphCast

Here, we introduce an MLWP approach for global medium-range weather forecasting called GraphCast, which produces an accurate 10-day forecast in under a minute on a single Google Cloud TPU (Tensor Processing Unit) v4 device and supports applications including predicting tropical cyclone tracks, atmospheric rivers, and extreme temperatures.

GraphCast takes as input the two most re-

Downloaded from https://www.science.org at University of Ba

Edit View History Bookmarks Tools Window Help
file:///Users/mascjb/Downloads/science.adi2336.pdf
1417 (2 of 6) 110%

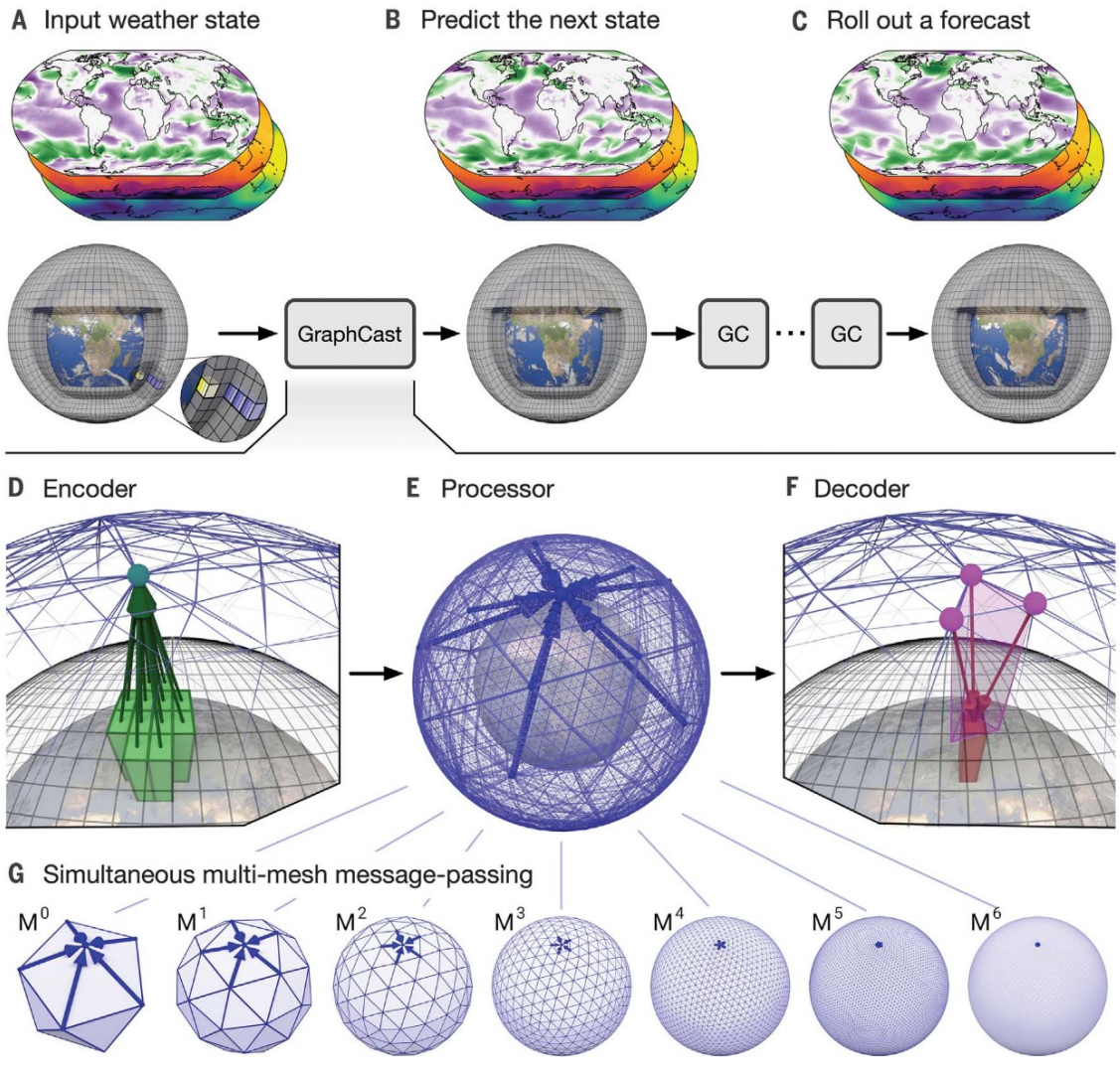


Fig. 1. Model schematic. (**A**) The input weather state(s) are defined on a 0.25° latitude-longitude grid comprising a total of 721 × 1440 = 1,038,240 points. Yellow layers in the close-up pop-out window represent the five surface variables, and blue layers represent the six atmospheric variables that are repeated at 37 pressure levels (5 + 6 × 37 = 227 variables per point in total), resulting in a state representation of 235,680,480 values. (**B**) GraphCast predicts the next state of the weather on the grid. (**C**) A forecast is made by iteratively applying GraphCast (GC) to each previous predicted state, to produce a sequence of states that represent the weather at successive lead times. (**D**) The encoder component of the GraphCast architecture maps local regions of the input (green boxes) into nodes of the multimesh graph representation (green, upward arrows that terminate in the green-blue node). (**E**) The processor component updates each multimesh node using learned message-passing (heavy blue arrows that terminate at a node). (**F**) The decoder component maps the processed multimesh features (purple nodes) back onto the grid representation (red, downward arrows that terminate

Downloaded from https://www.science.org at University of Bath on Ma

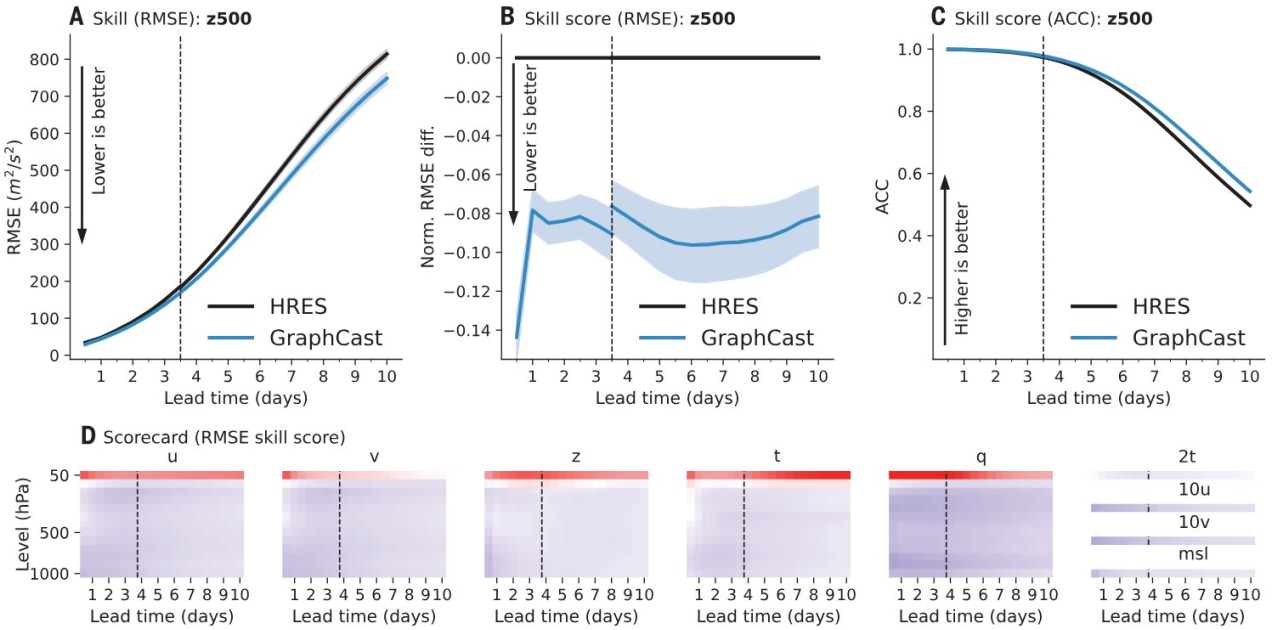


Fig. 2. Global skill and skill scores for GraphCast and HRES in 2018. (**A**) RMSE skill (*y* axis) for GraphCast (blue lines) and HRES (black lines), on *Z*500, as a function of lead time (*x* axis). Error bars represent 95% confidence intervals. The vertical dashed line represents 3.5 days, which is the last 12-hour increment of the HRES 06z/18z forecasts. The black line represents HRES, where lead times earlier and later than 3.5 days are from the 06z/18z and 00z/12z initializations, respectively. (**B**) RMSE skill score (*y* axis) for GraphCast versus HRES, on *Z*500, as a function of lead time (*x* axis). Error bars represent 95% confidence intervals for the skill score. We observe a discontinuity in GraphCast's curve because skill scores up to 3.5 days are computed between GraphCast (initialized at 06z/18z) and HRES's 06z/18z initialization, whereas skill scores after 3.5 days are computed with respect to HRES's 00z/12z initializations. (**C**) ACC skill (*y* axis) for GraphCast (blue lines) and HRES (black lines), on *Z*500, as a function of lead time (*x* axis). (**D**) Scorecard of RMSE skill scores for GraphCast, with respect to HRES. Each subplot corresponds to one variable: *U*, *V*, *Z*, *T*, *Q*, *2T*, *10U*, *10V*, and *MSL*. The rows of each heatmap correspond to the 13 pressure levels (for the atmospheric variables), from 50 hPa at the top to 1000 hPa at the bottom. The columns of each heatmap correspond to the 20 lead times at 12-hour intervals, from 12 hours on the left to 10 days on the right. Each cell's color represents the skill score, as shown in (B), where blue represents negative values (GraphCast has better skill) and red represents positive values (HRES has better skill).

vertical level (see supplementary materials eq. 19). | Of the 227 variable and level combinations | HRES forecasts against ERA5 would result in

Downloaded from https://www.science.org at University of Bath

transition from 06z/18z to 00z/12z initializations for HRES induced a small discontinuity in our plots, which is indicated by a vertically dashed line at the appropriate lead time. Supplementary materials section 5 contains further verification details, including details of the comparisons protocol between GraphCast and HRES (supplementary materials section 5.2) and the effect of initialization lookahead on both models' performance (supplementary materials section 5.2.2).

Forecast verification results

We find that GraphCast has greater weather forecasting skill than HRES when evaluated on 10-day forecasts at a horizontal resolution of 0.25° for latitude and longitude and at 13 vertical levels. Figure 2, A to C, shows how GraphCast (blue lines) outperforms HRES (black lines) on the $Z500$ (geopotential at 500 hPa) "headline" field in terms of RMSE skill, RMSE skill score [i.e., the normalized RMSE difference between model A and baseline B defined as $(RMSE_A - RMSE_B)/(RMSE_B)$], and ACC skill. Using $Z500$, which encodes the synoptic-scale pressure distribution, is common in the literature, as it has strong meteorological importance (*8*). The plots show that GraphCast has better skill scores across all lead times, with a skill score improvement of around 7 to 14%. Plots for additional headline variables are given in supplementary materials section 7.1.

Figure 2D summarizes the RMSE skill scores for all 1380 evaluated variables and pressure levels, across the 10-day forecasts, in a format analogous to the ECMWF Scorecard. The cell colors are proportional to the skill score, where blue indicates that GraphCast had better skill, and red indicates that HRES had better skill. GraphCast outperformed HRES on 90.3% of the

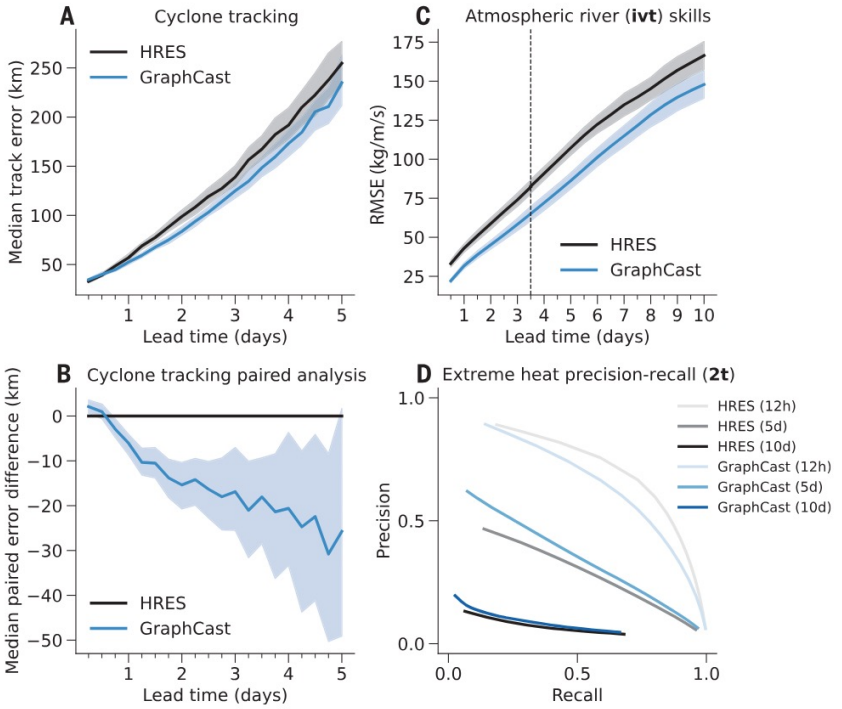


Fig. 3. Severe event prediction. (**A**) Cyclone tracking performances for GraphCast and HRES. The *x* axis represents lead times (in days), and the *y* axis represents median track error (in kilometers). Error bars represent bootstrapped 95% confidence intervals for the median. (**B**) Cyclone tracking paired error difference between GraphCast and HRES. The *x* axis represents lead times (in days), and the *y* axis represents median paired error difference (in kilometers). Error bars represent bootstrapped 95% confidence intervals for the median difference (see supplementary materials section 8.1). (**C**) Atmospheric river prediction (*IVT*) skills for GraphCast and HRES. The *x* axis represents lead times (in days), and the *y* axis represents RMSE. Error bars are 95% confidence intervals. (**D**) Extreme heat prediction precision-recall for GraphCast and HRES. The *x* axis represents recall, and the *y* axis represents precision. The curves represent different precision-recall trade-offs when sweeping over gain applied to forecast signals (see supplementary materials section 8.3).

Downloaded from https://www.science.org at University of Bath on

GraphCast's forecast skill and efficiency compared with HRES shows that MLWP methods are now competitive with traditional weather forecasting methods. Additionally, GraphCast's performance on severe event forecasting, which it was not directly trained for, demonstrates its robustness and potential for downstream value. We believe this marks a turning point in weather forecasting, which helps open new avenues to strengthen the breadth of weather-dependent decision-making by individuals and industries by making cheap prediction more accurate and accessible as well as suitable for specific applications. With 36.7 million parameters, GraphCast is a relatively small model by modern ML standards, chosen to keep the memory footprint tractable. And while HRES is released on 0.1 resolution, 137 levels, and up to 1-hour time steps, GraphCast operates on 0.25° latitude and longitude resolution, 37 vertical levels, and 6-hour time steps, because of the ERA5 training data's native 0.25° resolution and engineering challenges in fitting higher-resolution data on hardware. Generally, GraphCast should be viewed as a family of models, with the current version being the largest we can practically fit under current engineering constraints, but which have the potential to scale much further in the future with greater computer resources and higher-resolution data

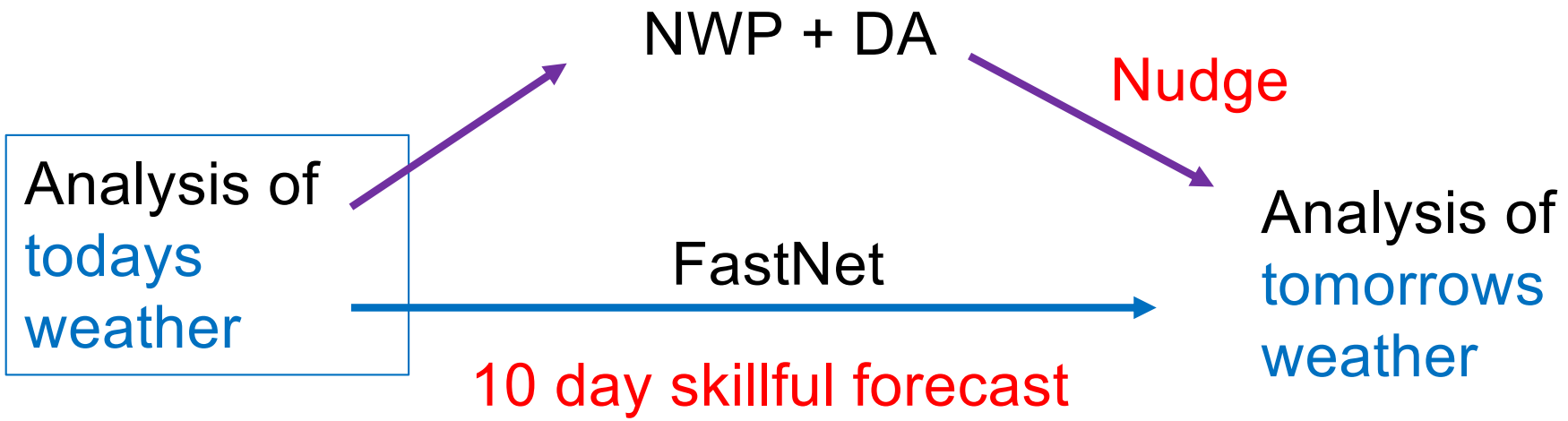
Criticisms of Graph-Cast

- Heavily based on a very skillfully created data set
- Makes predictions based on the data set, NOT on real (eg. satellite) data
- Suffers from spectral bias, so not resolving small scale phenomena well. Good for (smooth) pressure, not good for (localized) precipitation and fog.
- Norm used to compare with HRES is flattering to smoother predictions

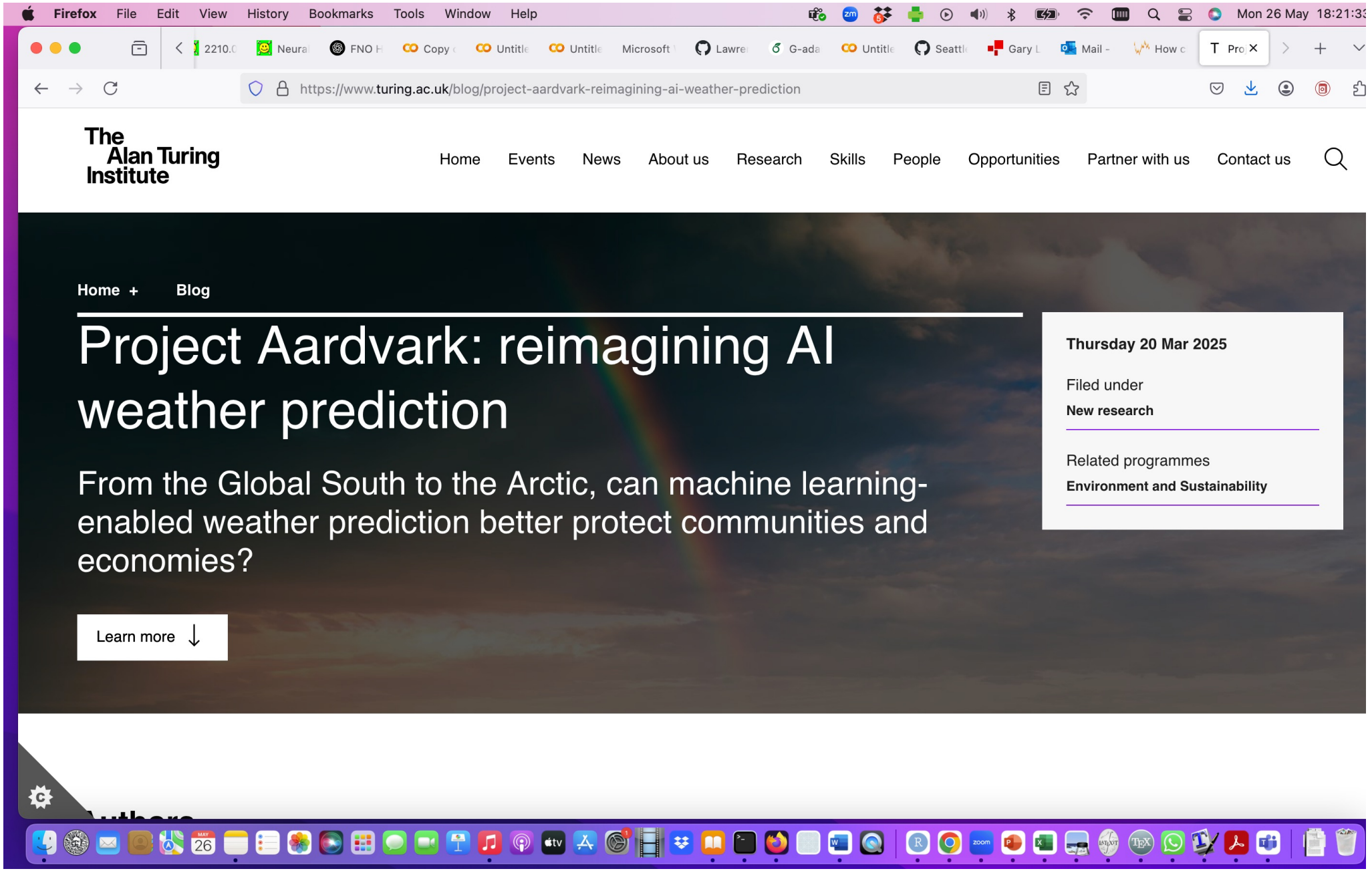
Predicts the wrong weather at the right time, rather than the right weather at the wrong time

Applications of GraphCast and FastNet (Met Office)

- Give good overall long time forecast quickly
- Predicts the climatology
- Can then be used to 'nudge' a high resolution forecast made by a numerical weather prediction physics based model (stabilizes/regularizes the forecast)



2025: Aardvark. Predictions from satellite data only



Assessing opportunities and challenges

Aardvark reimagines current weather prediction methods, offering a range of opportunities.

Alongside requiring less computing power, Aardvark is fast. Traditional forecasts can take hours to produce on a supercomputer whereas, once trained, Aardvark can create forecasts within minutes and can be run on a desktop computer.

There are highly promising signs of Aardvark's accuracy too. Globally, Aardvark is already as accurate as America's Global Forecast System (GFS), but it is only using about 10% of the available data to make its forecasts, meaning that further improvements in accuracy should be possible. We are excited to see what happens as we increase the amount of data and optimise Aardvark end-to-end to provide more accurate forecasts. This new paradigm could replace the traditional numerical approach in developing countries.

A streamlined system like this could also play a significant role in democratising access to advanced forecasting tools, empowering developing or data-sparse countries to build capacity and create bespoke weather forecasting systems that previously would have required large teams to operate, deploy and maintain.

There are of course challenges and it's important to acknowledge that, whilst machine learning weather tools are moving at great pace, this is still an experimental technology that will require rigorous evaluation over a period of time.

Weather prediction tools must accurately predict all types of weather, and extremes like hurricanes and floods are especially important. Unfortunately, rare events like these are less represented in the training data, meaning that AI systems may struggle more on these phenomena.

We also need to ensure we account for our changing climate, which could render models trained on past data less accurate.

Early signs suggest that we can rise to these challenges.

What's next for Aardvark?

file:///Users/mascjb/Downloads/s41586-025-08897-0.pdf
of 15
Automatic Zoom

Article

End-to-end data-driven weather prediction

https://doi.org/10.1038/s41586-025-08897-0

Received: 10 July 2024

Accepted: 12 March 2025

Published online: 20 March 2025

Open access

Check for updates

Anna Allen[1,11]✉, Stratis Markou[2,11]✉, Will Tebbutt[2,9], James Requeima[3], Wessel P. Bruinsma[4], Tom R. Andersson[5,10], Michael Herzog[6], Nicholas D. Lane[1], Matthew Chantry[7], J. Scott Hosking[5,8] & Richard E. Turner[2,8]✉

Weather prediction is critical for a range of human activities, including transportation, agriculture and industry, as well as for the safety of the general public. Machine learning transforms numerical weather prediction (NWP) by replacing the numerical solver with neural networks, improving the speed and accuracy of the forecasting component of the prediction pipeline[1–6]. However, current models rely on numerical systems at initialization and to produce local forecasts, thereby limiting their achievable gains. Here we show that a single machine learning model can replace the entire NWP pipeline. Aardvark Weather, an end-to-end data-driven weather prediction system, ingests observations and produces global gridded forecasts and local station forecasts. The global forecasts outperform an operational NWP baseline for several variables and lead times. The local station forecasts are skilful for up to ten days of lead time, competing with a post-processed global NWP baseline and a state-of-the-art end-to-end forecasting system with input from human forecasters. End-to-end tuning further improves the accuracy of local forecasts. Our results show that skilful forecasting is possible without relying on NWP at deployment time, which will enable the realization of the full speed and accuracy benefits of data-driven models. We believe that Aardvark Weather will be the starting point for a new generation of end-to-end models that will reduce computational costs by orders of magnitude and enable the rapid, affordable creation of customized models for a range of end users.

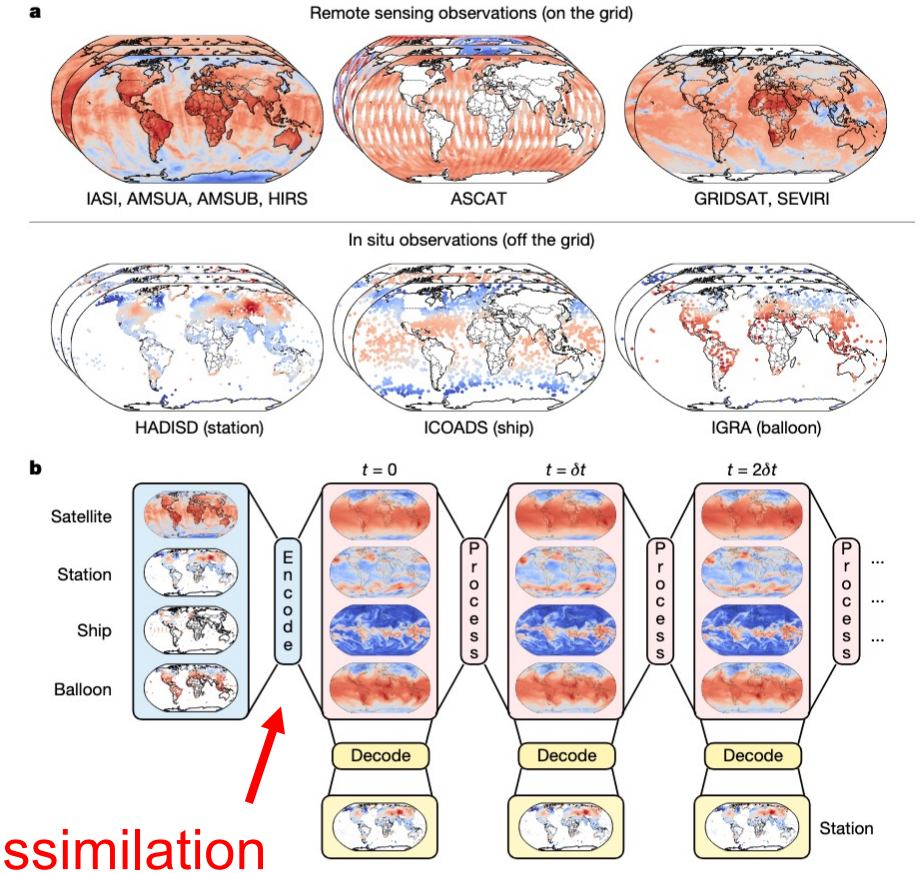


Fig. 1 | Data and operation of Aardvark Weather. a, Different data sources leveraged in Aardvark. The input data consist of observations from remote sensing instruments (top row), which we pre-grid before passing to the model, as well as in situ observations from land and marine observation platforms and radiosondes (bottom row). Each of these data modalities contains several observational variables, of which we selected a subset here for the purposes of illustration. Here we show remote sensing data[40–45], after performing our gridding step, and raw in situ data[46–48]. Note that the colours in all six plots are meant for illustration purposes. The remote sensing data also include a range of metadata about the measurements, omitted here for simplicity. White areas indicate regions of missing data, which must be handled by the encoder module of Aardvark. **b**, Aardvark at deployment time. First, an encoder module uses raw observations as input to estimate the initial state of the atmosphere across key variables at $t = 0$. Next, a processor module ingests the estimated state to produce a forecast at the next lead time $t = \delta t$. Forecasts at subsequent lead times are produced autoregressively. Finally, a decoder module is applied to the on the grid states to produce off the grid predictions. The modular design of Aardvark allows for pretraining on large high-quality ERA5 reanalysis data[34]. In this figure, the displayed data are the training data used to train each module of Aardvark from the aforementioned sources.

file:///Users/mascjb/Downloads/s41586-025-08897-0.pdf
3 of 15
100%

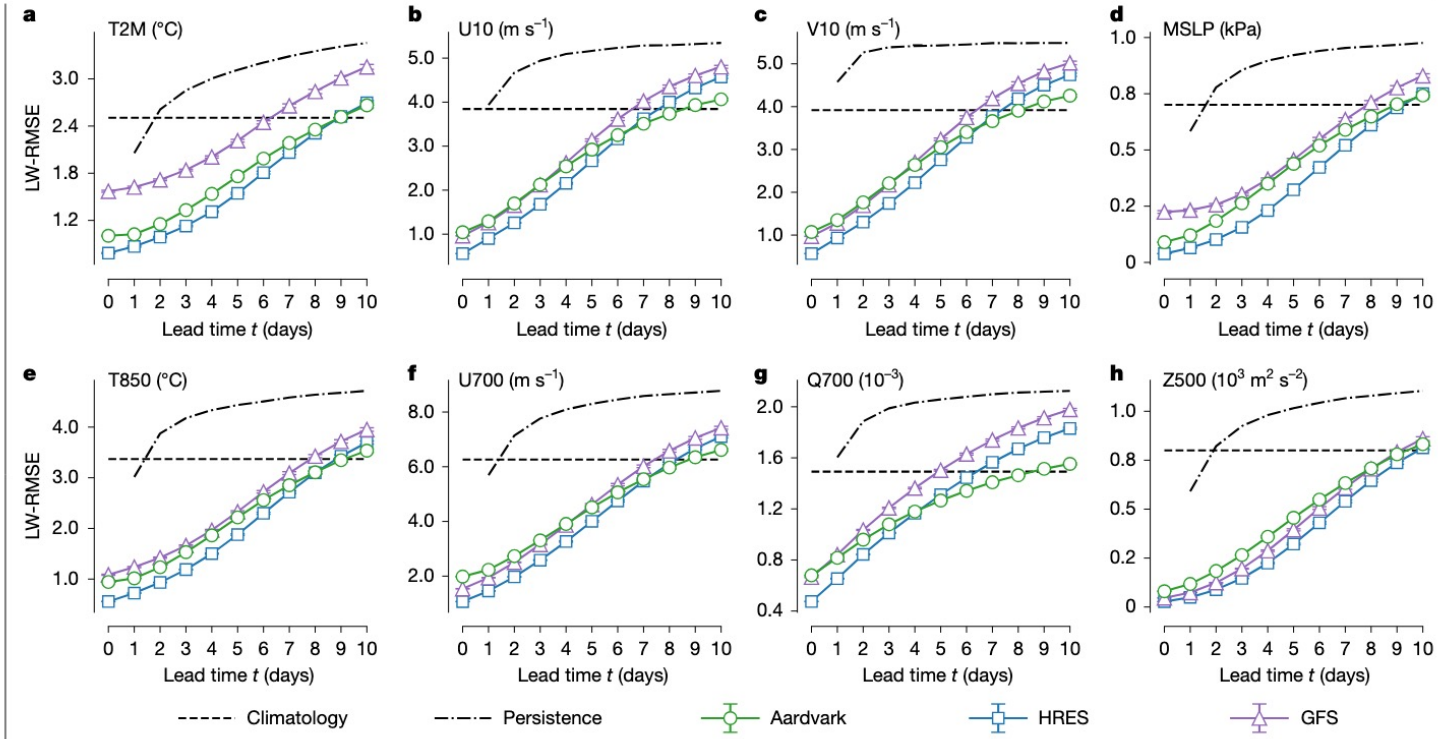


Fig. 2 | Gridded global forecast performance for selected variables. **a**–**h**, Latitude-weighted RMSE using ERA5 (ref. 34) reanalysis data as the ground truth, on the held-out test year (2018), for the four surface variables: 2-m temperature (**a**; T2M), 10-m eastward wind (**b**; U10), 10-m northward wind (**c**; V10) and mean sea level pressure (**d**; MSLP), as well as four headline upper-atmosphere variables: temperature at 850 hPa (**e**; T850), eastward wind at 700 hPa (**f**; U700), specific humidity at 700 hPa (**g**; Q700) and geopotential at 500 hPa (**h**; Z500) as a function of lead time t. At lead time $t = 0$, Aardvark predicted the initial atmospheric state from observational data alone. The error at $t = 0$ corresponds to the error in the initial state. Note that HRES has a non-zero error at $t = 0$ compared to ERA5 reanalysis. The HRES forecasts[33] we used have been conservatively re-gridded to prevent aliasing, and we performed the same operation on the GFS forecasts[49]. We report the mean performance of each system together with 98% confidence intervals in our estimate of the mean performance.

use a recurrent update in which the previous forecast is adjusted in light of new observations, similar to Kalman filter recursions in a Markov model. In principle, data assimilation accumulates information from observations across all past time steps. However, in practice, it has been estimated that the effective window size is as short as 4 days (ref. 25). Owing to the complexities of training recurrent neural networks, including the need for a spin-up period and gradient instabilities[26], we opted a way that mimics how it will be deployed. We started by pretraining the encoder module using raw observations as input and reanalysis data as targets. An advantage of this machine learning approach is that the model can learn to correct for biases in the input observations during training; therefore, no bias correction step was performed on the input data. We also pretrained the processor using reanalysis data for both inputs and targets and then fine-tuned the output of the

ML Parametrisations

Augment a NWP forecast by using ML to give sub-grid scale level physics



Example: CARAMEL Project: Cloud fraction prediction

Scale-Aware Parameterization of Cloud Fraction and Condensate for a Global Atmospheric Model Machine-Learned From Coarse-Grained Kilometer-Scale Simulations

Cyril Morcrette, Tobias Cave, Helena Reid, Joana da Silva Rodrigues, Teo Deveney, Lisa Kreusser, Kwinten Van Weverberg, and B

Conclusions

Scientific Machine Learning is showing great promise in both making its own weather forecasts and in enhancing existing forecasting methods

We will have to see if the great recent claims are fully justified

This is a VERY rapidly growing field

Watch this space!!!!



